

KRAVSPECIFIKATION – AKR

KRAVSPECIFIKATION – AKR.....	1
1. Indledning.....	2
1.1 Formål.....	2
1.2 Referencer.....	2
1.3 Læsevejledning.....	2
2. Generel beskrivelse.....	2
2.1 Systembeskrivelse.....	2
2.2 Systemets funktion.....	2
2.3 Systembegrænsninger.....	3
2.4 Systemets fremtid.....	3
2.5 Brugerprofil.....	3
2.6 Løsningsoversigt.....	3
2.7 Krav til udviklingsforløbet.....	4
3. Specifikke krav.....	5
3.1 Funktionelle krav.....	5
Gameplay.....	5
Farming – Plantning.....	5
Farming – Høst.....	5
Ressourcesamling.....	5
DyreSystem – Jagt.....	5
Hunger system.....	5
Exploration.....	5
3.2 Informationsmodel.....	6
3.3 Procesmodel.....	6
3.4 Løsningsarkitektur.....	6
3.5 Prioritering (MoSCoW).....	6
3.6 Acceptkriterier.....	7
3.7 Use Cases.....	9
3.8 Wireframes.....	14
4. Eksterne grænseflader.....	14
4.1 Brugergrænseflade.....	14
4.2 Hardware.....	14
4.3 Software.....	14
5. Krav til ydelse.....	15
6. Kvalitetsfaktorer.....	15
Pålidelighed.....	15
Vedligeholdelse.....	15
UdvidelsesVenlighed.....	15
Brugervenlighed.....	15
Designprincip.....	16

1. Indledning

1.1 Formål

Formålet med dette projekt er at udvikle spillet AKR, et cozy farming-spil udviklet i Unity, hvor spilleren kan udforske, samle ressourcer og opbygge progression i et nordisk inspireret univers. Produktet skal levere en afslappende spiloplevelse med fokus på modulær opbygning, stabilitet og udvidelsesmuligheder. Systemet designes med henblik på at kunne understøtte én eller flere spillere afhængigt af projektets endelige scope og tidsramme.

1.2 Referencer

- Ønsket om at spillet skal være i 2D kræver en optimeret game engine til den type spil, af denne grund har vi valgt Unity som hoved værktøj frem for Unreal Engine eller andre alternativer, [da Unity er bedre udstyret](#) til at behandle denne type af spil.
- Vores valg af programmeringssprog er C#. Grunden til dette er, at Unity er tæt integreret med C#, som bruges til at styre spillets logik, objekter og adfærd. Dette gør sproget til et naturligt valg for os. Derudover ligger sproget tæt op ad vores faglige profil, da skolens undervisningsforløb i høj grad er opbygget omkring C#.

1.3 Læsevejledning

Dokumentet beskriver først systemets overordnede funktion og begrænsninger, derefter en løsningsoversigt samt konkrete funktionelle krav og kvalitetskrav. Dokumentet inkluderer desuden informationsmodel, procesmodel, løsningsarkitektur, prioritering (MoSCoW) og acceptkriterier.

2. Generel beskrivelse

2.1 Systembeskrivelse

AKR er et pixelart spil udviklet i Unity med C#. Spilleren interagerer via mus og tastatur og styrer sin progression gennem farming, exploration, ressourcehåndtering og jagt på vilde dyr. Spillet designes primært omkring en spilleroplevelse, men systemarkitekturen opbygges modulært med henblik på mulig udvidelse til multiplayer.

2.2 Systemets funktion

Spillet skal give spilleren mulighed for at: dyrke afgrøder, samle ressourcer, jage vilde dyr, udforske områder, håndtere inventory og opleve et day/night-system. Spillet skal desuden understøtte et afgrænset "basic building/placement", hvor spilleren kan placere simple objekter i verdenen.

2.3 Systembegrænsninger

Spillet fokuserer på afslappende gameplay uden strafmekanikker. Følgende funktioner er udeladt i denne iteration: online services (fx accounts, matchmaking, chat og dedicated server-drift).

Multiplayer: Multiplayer er ikke et baselinekrav i denne iteration og indgår derfor ikke i Must-kravene. Multiplayer kan implementeres som optional funktion, afhængigt af tid og ressourcer (se MoSCoW).

2.4 Systemets fremtid

Systemet designes modulært med henblik på udvidelse og nem tilføjelse af nye mekanismer eller spilprincipper. Mulige fremtidige udvidelser kan omfatte peer-to-peer multiplayer, flere biomes med unikke farver, planter og crops, flere dyr og crops i forskellige varianter samt et mere avanceret byggesystem.

2.5 Brugerprofil

Systemet henvender sig til flere brugergrupper med forskellige forudsætninger og adgangsformer.

Brugergrupper

- **Primær brugergruppe – Spillere:** Casual spillere som ønsker en afslappende spiloplevelse med fokus på farming, exploration og progression.
- **Sekundær brugergruppe – Udviklere:** Udviklere som arbejder videre på projektet i Unity og vedligeholder systemets funktioner og struktur.

Brugernes forudsætninger

- **Spillere** forventes ikke at have teknisk erfaring og skal kunne navigere spillet via intuitive kontroller og UI.
- **Udviklere** forventes at have grundlæggende kendskab til Unity, C# og modulær systemopbygning.

Adgangsform

- Spillere tilgår systemet via spillets brugergrænseflade med mus og tastatur på PC.
- Udviklere tilgår projektet gennem Unity Editor samt versionsstyringssystemet.

2.6 Løsningsoversigt

AKR består af flere gameplay-systemer, som tilsammen udgør spillets kerneoplevelse: farming, inventory, exploration, ressourcehåndtering, day/night og basic building/placement. Systemerne designs modulært, så funktionalitet kan udvikles og udvides uafhængigt.

Spillets gameplay-loop består overordnet af, at spilleren udforsker verdenen og samler ressourcer, anvender værktøjer til farming og basic placement, gemmer ressourcer i inventory og opnår progression gennem arbejde og planlægning.

2.7 Krav til udviklingsforløbet

Projektets udviklingsforløb skal følge faste tekniske og dokumentationsmæssige rammer for at sikre struktur, sporbarhed og kvalitet gennem hele udviklingsperioden.

Udviklingsplatform

- Projektet skal udvikles i Unity Game Engine ved brug af C#.
- Systemerne skal opbygges modulært i overensstemmelse med løsningsarkitekturen.

Versionsstyring

- Versionsstyring skal anvendes via Unitys indbyggede cloud-løsning.
- Ændringer i funktionalitet og nye implementationer skal kunne spores gennem versionshistorik.

Logbog og dokumentation

- Alle væsentlige ændringer og nye features skal dokumenteres i logbogen med korrekt dato og tidspunkt.
- Under udviklingsperioden skal der udarbejdes en teknisk rapport, som beskriver:
 - Spillets tekniske opbygning
 - Systemarkitektur og centrale systemer
 - Centrale scripts og kodeprincipper

Diagramkrav

Projektet skal indeholde følgende dokumentation:

- Kravspecifikation og designbeskrivelse
- Use Cases for centrale gameplay-funktioner
- E/R-diagram for inventory- og datastruktur

3. Specifikke krav

3.1 Funktionelle krav

Gameplay

Farming – Plantning

- Systemet skal tillade plantning af et seed-item fra Inventory.
- Plantning må kun ske på gyldige jord-tiles i verdenen.
- Spilleren skal have mindst ét seed-item i Inventory for at kunne plante.
- Ved plantning skal seed-item fjernes fra Inventory.
- Systemet skal oprette et Crop-objekt med initialt vækststadie.

Farming – Høst

- Høst må kun ske når Crop har nået fuldt vækststadie.
- Ved høst skal Crop-objektet fjernes fra verdenen.
- Systemet skal tilføje korrekt item-type og mængde til Inventory.
- UI skal opdateres efter ændring i Inventory.

Ressourcesamling

- Systemet skal registrere interaktion mellem Player og WorldObject.
- Ved gyldig interaktion skal korrekt item tilføjes Inventory.
- WorldObject skal fjernes fra verdenen efter pickup.
- UI skal opdateres ved ændring i Inventory.

DyreSystem – Jagt

- Spilleren skal kunne angribe dyr i verdenen.
- Når dyrets health når 0, skal dyret fjernes fra verdenen.
- Dyret skal generere et loot-item (kød).
- Loot-item skal kunne samles op og tilføjes Inventory.

Hunger system

- Spilleren skal kunne spise items fra Inventory for at opnå en midlertidig buff.
- Systemet må ikke indeholde strafmekanik ved manglende spisning.

Exploration

- Systemet skal tillade Player at bevæge sig frit inden for world boundaries.

- Player skal kunne interagere med objekter i verdenen, såsom ressourcer, dyr og placerbare objekter.
- Player skal kunne samle ressourcer fra verdenen gennem interaktion.
- Player skal kunne møde vilde dyr i verdenen som kan nedlægges for ressourcer.
- Verdenen skal indlæses korrekt ved scene- eller områdeovergang.

3.2 Informationsmodel

Systemet baseres på følgende centrale informationsenheder: Player, Inventory, Item, Crop, WorldObject og TimeState.

Player har et Inventory. Inventory indeholder Items. Crop repræsenterer en plantet afgrøde med vækststadie og høst-output. TimeState repræsenterer tidsprogression (fx dag/nat) og påvirker logik såsom crop-vækst. WorldObject repræsenterer placerbare objekter fra building/placement-systemet.

3.3 Procesmodel

Et generelt flow for interaktion er: Input → handling → systemopdatering → UI feedback.

Farming-flow: Spilleren vælger seed i inventory, planter i verdenen, time/day-night opdaterer vækststadie, spilleren høster, inventory opdateres, og UI viser ændringen.

Resource pickup-flow: Spilleren interagerer med en ressource i verdenen, et item tilføjes til inventory, og UI opdateres.

Jagt-flow: Spilleren møder et dyr i verdenen, angriber dyret, dyrets health reduceres, og når health når 0 fjernes dyret fra verdenen. Systemet genererer et loot-item (kød), som spilleren kan samle op og få tilføjet til inventory. UI opdateres efter ændringen.

3.4 Løsningsarkitektur

Løsningen opdeles i løst koblede moduler: Player Controller, Farming System, Animal System, Inventory System, Time System, Building/Placement System og UI Layer. Modulerne kommunikerer via klare grænseflader, så ændringer i ét system i begrænset omfang påvirker de øvrige.

Optional modul: Networking Module kan tilføjes for multiplayer.

3.5 Prioritering (MoSCoW)

MoSCoW-prioriteringen er styrende for projektets scope. Prioriteringen er gjort konsistent med system begrænsningerne, således at funktioner markeret som "Udeladt/Excluded" ikke samtidig optræder som Must/Should/Could/Would.

Feature	Status	Noter
Farming (plant/høst)	Must	Kerne i cozy farming-loop.
Ressourcesamling	Must	Pickup/Harvest giver items.
Dyresystem (jagt)	Must	Spilleren kan nedlægge vilde dyr for ressourcer
Exploration	Must	Udforskning af områder.
Inventory + UI visning	Must	Håndtering og feedback til spiller.
Day/Night-system	Must	Tidsprogression påvirker verden/logik.
Basic building/placement	Must	Afgrænset til simple objekter.
Multiplayer (netværk)	Could	Optional, afhænger af tid og netværksmodel
Peer-to-peer multiplayer	Would	Perspektivering; ikke baseline.
Flere biomes/variationer	Would	Perspektivering; ikke baseline.
Flere dyr og crop-varianter	Would	Perspektivering; ikke baseline.
Basic fishing system	Would	Perspektivering; ikke baseline.
Avanceret byggesystem	Would	Perspektivering; ud over "basic".
Tool progression	Would	Perspektivering; ikke baseline

3.6 Acceptkriterier

Kravene anses for opfyldt når følgende kan verificeres i en testsession:

Farming: Spilleren kan plante, afgrøden ændrer vækststadie via time/day-night, og høst tilføjer korrekt item til inventory samt opdaterer UI.

Ressourcesamling: Interaktion med en ressource i verdenen tilføjer korrekt item/mængde til inventory og opdaterer UI uden fejl.

Dyresystem: Spilleren kan nedlægge et dyr, og systemet genererer korrekt loot som tilføjes inventory.

Exploration: Spilleren kan bevæge sig og interagere med verdenens elementer i mindst ét område uden fejl.

Inventory: Inventory kan åbnes/lukkes, viser korrekt indhold, og opdateres korrekt ved harvest/pickup.

Day/Night: Systemet skifter mellem mindst to faser (dag/nat) og påvirker spillets tilstand (fx visuel tilstand og/eller crop-vækst).

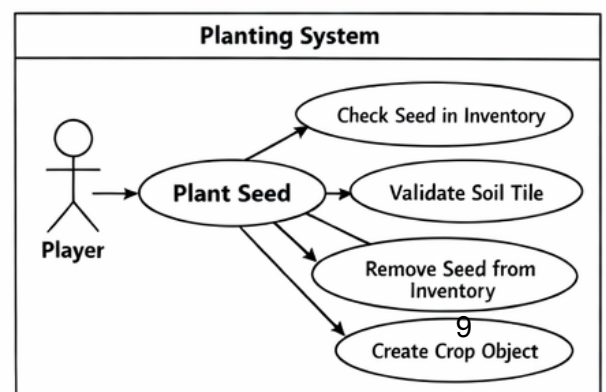
Basic building/placement: Spilleren kan placere mindst én type objekt; objektet vises korrekt og kan ikke placeres på ugyldige steder (afgrænses i implementering).

Stabilitet: Spillet kan køre i minimum 300 minutter ved normal brug uden crash.

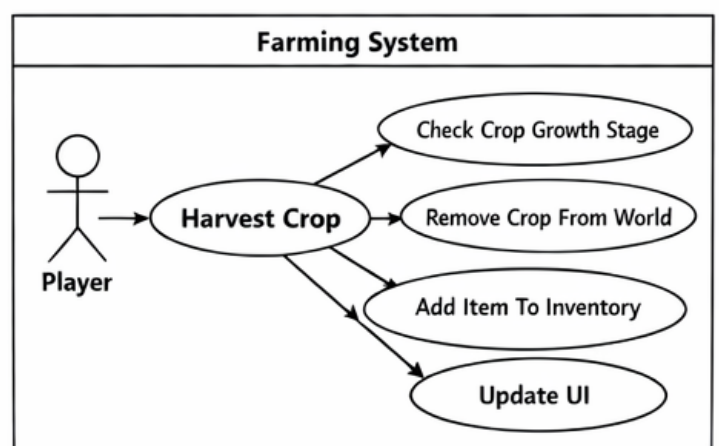
Ydelse: Kravene i kapitel 5 anses for opfyldt når spillet opretholder minimum 60 FPS under test.

3.7 Use Cases

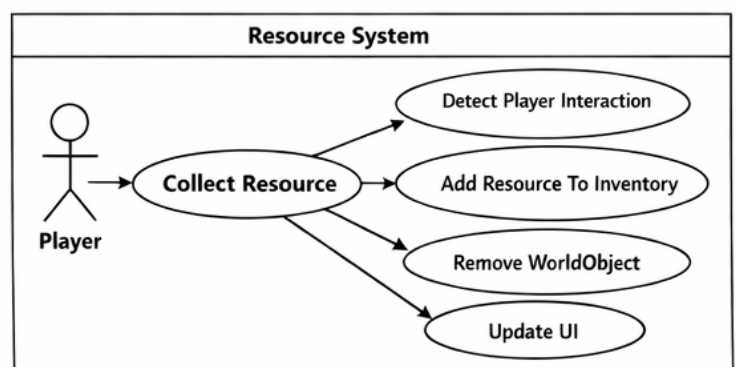
Felt	Indhold
Use case ID	UC-01
Navn	Plant seed (Plantning)
Primær aktør	Spiller
Formål	At plante et seed-item fra Inventory på en gyldig jord-tile.
Preconditions	(1) Spilleren har mindst 1 seed-item i Inventory. (2) Spilleren står ved / har valgt en gyldig jord-tile .
Trigger	Spilleren vælger seed i Inventory og udfører "Plant" på en tile.
Hovedforløb	1) Spiller vælger et seed-item i Inventory. 2) Spiller vælger en tile i verdenen. 3) System validerer at tile er gyldig jord. 4) System validerer at seed findes i Inventory (antal ≥ 1). 5) System fjerner 1 seed-item fra Inventory. 6) System opretter et Crop-objekt på tile med initialt vækststadie. 7) UI opdateres (Inventory).
Alternative flows / Exceptions	A1: Tile er ikke gyldig jord → System afviser plantning, ingen ændringer. A2: Ingen seeds i Inventory → System afviser plantning, ingen ændringer.
Postconditions	Crop findes i verdenen i initial vækststadie, og Inventory er reduceret med 1 seed-item.



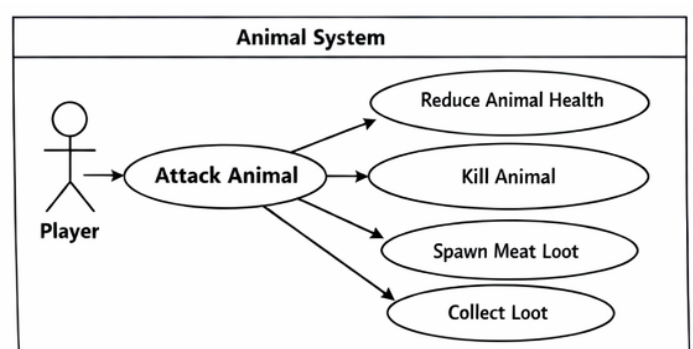
Felt	Indhold
Use case ID	UC-02
Navn	Harvest crop (Høst)
Primær aktør	Spiller
Formål	At høste en fuldt udvokset Crop og få items i Inventory.
Preconditions	(1) Crop findes i verdenen. (2) Crop er i fuldt vækststadie .
Trigger	Spilleren interagerer med en Crop og vælger "Harvest".
Hovedforløb	1) Spiller interagerer med Crop. 2) System validerer at Crop er fuldt udvokset. 3) System fjerner Crop-objektet fra verdenen. 4) System tilføjer korrekt item-type og mængde til Inventory. 5) UI opdateres (Inventory).
Alternative flows / Exceptions	A1: Crop er ikke fuldt udvokset → System afviser høst, ingen ændringer.
Postconditions	Crop er fjernet fra verdenen, og Inventory indeholder høst-output.



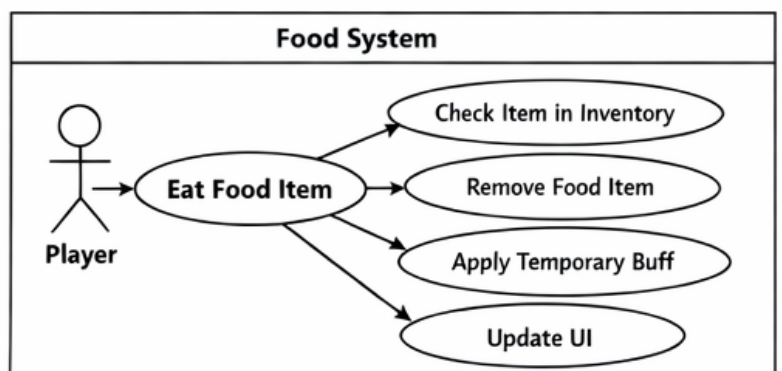
Felt	Indhold
Use case ID	UC-03
Navn	Collect resource (Ressourcesamling)
Primær aktør	Spiller
Formål	At samle en ressource (WorldObject) op og få item i Inventory.
Preconditions	(1) WorldObject findes i verdenen. (2) Spilleren kan interagere (inden for rækkevidde/gyldig interaktion).
Trigger	Spilleren interagerer med et WorldObject (fx "Pick up").
Hovedforløb	1) Spiller interagerer med WorldObject. 2) System registrerer Player–WorldObject interaktion. 3) System validerer at interaktionen er gyldig. 4) System tilføjer korrekt item (type + mængde) til Inventory. 5) System fjerner WorldObject fra verdenen. 6) UI opdateres (Inventory).
Alternative flows / Exceptions	A1: Interaktion ikke gyldig (for langt væk / blokeret) → System afviser pickup, ingen ændringer.
Postconditions	WorldObject er fjernet, og Inventory er opdateret med ressourcen.



Felt	Indhold
Use case ID	UC-04
Navn	Hunt animal (Jagt)
Primær aktør	Spiller
Formål	At nedlægge et dyr og få loot (kød) i Inventory.
Preconditions	(1) Et dyr findes i verdenen. (2) Spilleren kan angribe dyret (inden for rækkevidde/angrebsinput muligt).
Trigger	Spilleren udfører "Attack" på et dyr.
Hovedforløb	1) Spiller angriber dyret. 2) System reducerer dyrets health. 3) Når health når 0: system fjerner dyret fra verdenen. 4) System genererer loot-item (kød) i verdenen. 5) Spiller samler loot op (interaktion). 6) System tilføjer loot til Inventory. 7) UI opdateres (Inventory).
Alternative flows / Exceptions	A1: Dyret dør ikke (health > 0) → Use case fortsætter ved næste angreb. A2: Loot kan ikke samles (interaktion mislykkes) → Loot forbliver i verdenen, ingen Inventory-ændring.
Postconditions	Dyret er fjernet, og loot er enten i Inventory (hvis samlet op) eller ligger i verdenen.



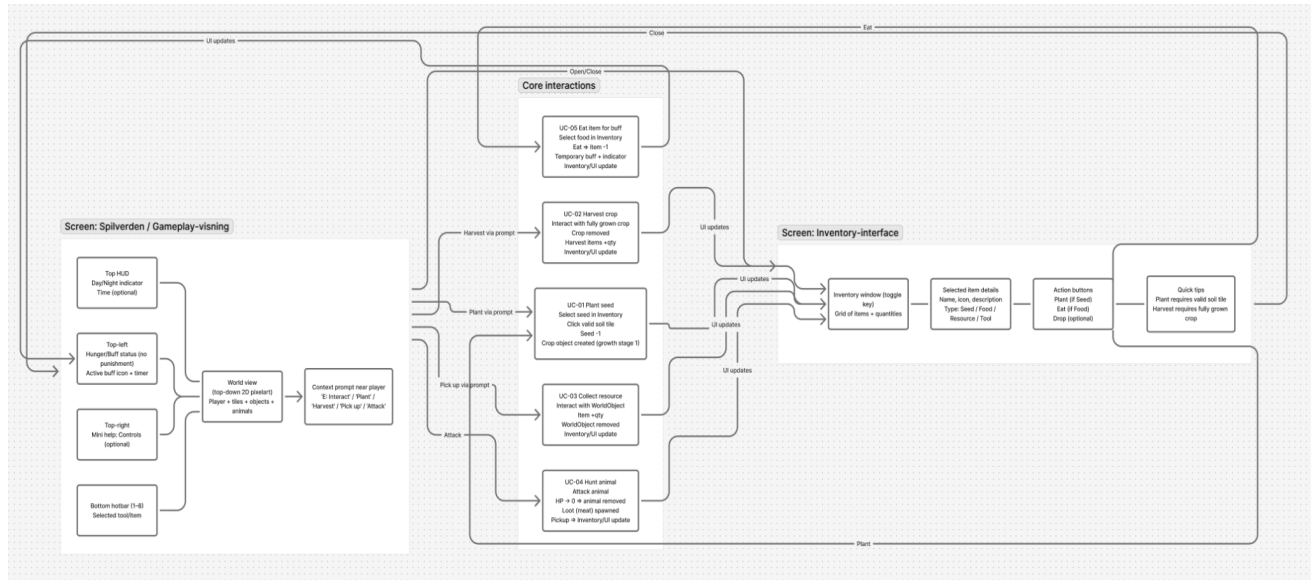
Felt	Indhold
Use case ID	UC-05
Navn	Eat item for buff (Spis for buff)
Primær aktør	Spiller
Formål	At spise et item fra Inventory og få en midlertidig buff.
Preconditions	(1) Spilleren har et spiseligt item i Inventory (antal ≥ 1).
Trigger	Spilleren vælger "Eat/Consume" på et item i Inventory.
Hovedforløb	1) Spiller åbner Inventory. 2) Spiller vælger et spiseligt item og vælger "Eat". 3) System validerer at item findes i Inventory og er spiseligt. 4) System fjerner 1 item fra Inventory. 5) System giver spilleren en midlertidig buff (defineret varighed/effekt). 6) UI opdateres (Inventory + evt. buff-indikator).
Alternative flows / Exceptions	A1: Item findes ikke / antal = 0 $\rightarrow$ System afviser. A2: Item er ikke spiseligt $\rightarrow$ System afviser.
Postconditions	Buff er aktiv i en periode, og Inventory er reduceret med 1. Ingen strafmekanik udløses ved ikke at spise.



3.8 Wireframes

wireframes indgår:

- Spilverden (gameplay-visning)



5. Krav til ydelse

- Spillet skal opretholde minimum 60 FPS under normal gameplay på projektets testmaskine.
 - Frame drops må ikke forekomme kontinuerligt under almindelig interaktion.
 - Input-respons må ikke overstige 100 ms ved normal brug.
-

6. Kvalitetsfaktorer

Pålidelighed

- Systemet skal kunne køre minimum 300 minutter under normal gameplay uden runtime-fejl eller crash.

Vedligeholdelse

- Systemet skal være opdelt i selvstændige moduler med klare ansvarsområder.
- Ændringer i ét gameplay-system må ikke kræve omskrivning af øvrige kernekomponenter.

UdvidelsesVenlighed

- Nye crops, items og world-objekter skal kunne tilføjes uden ændring af eksisterende logik.
- Arkitekturen skal understøtte tilføjelse af networking-modul uden fuld systemomstrukturering.

Brugervenlighed

- Grundlæggende funktioner (plantning, høst, inventory) skal kunne anvendes uden ekstern vejledning.
- UI skal tydeligt indikere ændringer i inventory og ressourcer.

Designprincip

- Spillet må ikke indeholde permanente strafmekanikker som tab af progression eller ressourcer.